PROJECT REPORT ON

Simple Drawing App

Submitted by:
Name: Asutosh Sahu
Roll No: 03BCA2514

Under the Guidance Of:
Mr. Saumya Ranjan Pradhan
Department of Computer Application

July, 2026

DEPARTMENT OF COMPUTER APPLICATION

CREATIVE TECHNO COLLEGE
BALUAKATA, SARADHAPUR, ANGUL

STUDENT DECLARATION

I hereby declare that the project report entitled **"Simple Drawing App"** submitted in partial fulfillment of the requirement for the degree of BCA to Utkal University. This project is not submitted to any other university for the award of any other degree, fellowship or any other similar title of prizes.

Place: Baluakata, Saradhapur, Angul **Name of Candidate:** Asutosh Sahu

Date: **Roll. No.:** 03BCA2514

CREATIVE
TECHNO COLLEGE
ANGUL

CERTIFICATE

This is to certify that Mr. **ASUTOSH SAHU** a student of BCA First Semester bearing roll no: **03BCA2514** has successfully completed the Bonafide project entitled **Simple Drawing App**. This project report is being accepted as the fulfillment of the requirement for the award degree of Bachelor of Computer Application at Creative Techno College under Utkal University, Odisha.

Signature **Signature**

(Head of the Department) **Internal Guide**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who contributed to the successful completion of this project.

I am especially thankful to my respected guide, **Mr. Saumya Ranjan Pradhan**, for his valuable guidance, continuous support, and encouragement throughout the project. His suggestions and expertise were invaluable in completing this work successfully.

I also extend my heartfelt thanks to **Mr. Swastik Ranjan Sahoo**, Project Supervisor, for his guidance, motivation, and constant support during the project.

I sincerely thank our **Dean (Academic), Principal, Vice Principal, and all the teachers** for their encouragement, guidance, and continuous support throughout our academic journey.

Finally, I express my gratitude to the **faculty members and staff of Creative Techno College, Angul**, as well as everyone who directly or indirectly contributed to the successful completion of this project.

Thank you.

ABSTRACT

The **Simple Drawing App** is a desktop-based graphical application developed using **Python**, **Tkinter**, and **Pillow (PIL)** that provides a simple and interactive digital drawing environment. The application enables users to create freehand drawings using a **Pencil**, erase unwanted portions with an **Eraser**, and draw different geometric shapes such as **Lines, Rectangles, Circles, and Triangles** through a convenient **Shapes dropdown menu**. Users can customize their drawings by selecting different pen colors, changing the canvas background color, and adjusting the brush size using a vertical slider. The application also provides options to clear the drawing canvas and save the completed artwork as **PNG** image files.

The project demonstrates the practical implementation of **Graphical User Interface (GUI) design**, **event-driven programming**, **mouse event handling**, **computer graphics**, **file handling**, and **basic image processing** concepts. It also incorporates modern interface elements such as a structured toolbar, status bar, and color selection tools to improve user experience. With its simple, interactive, and user-friendly interface, the application serves as an excellent learning project for beginners in Python GUI development while providing hands-on experience in integrating graphical components with real-time drawing functionalities.

TABLE OF CONTENTS

No.	Topics	Page
1.	Introduction	1
2.	Project Overview	2
3.	Objective	3
4.	Software & Hardware Requirements	4
5.	Process	5 – 11
6.	Source Code	12 – 22
7.	Output	23 – 30
8.	Conclusion	31
9.	Future Scope	32
10.	Reference	33

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.	Initial Screen	23
2.	Freehand Drawing	24
3.	Geometric Shapes	25
4.	Eraser Tool	26
5.	Changing Color	27
6.	Brush Size and Clearing	28
7.	Saving	29
8.	Status Bar	30

INTRODUCTION

Drawing applications are widely used for creating sketches, diagrams, and simple artwork through an interactive graphical interface. They provide an easy way to express ideas visually and are useful in education, design, and personal projects.

The **Simple Drawing App** is a desktop-based application developed using **Python**, **Tkinter**, and **Pillow (PIL)**. It allows users to create freehand drawings using a **Pencil**, erase unwanted parts with an **Eraser**, draw **Lines, Rectangles, Circles, and Triangles**, choose pen and background color, adjust the brush size, clear the canvas, and save drawings as **PNG** images. The application also includes a status bar that displays the selected tool, brush size, and pen color.

This project demonstrates the concepts of **GUI design**, **event-driven programming**, **mouse event handling**, **canvas graphics**, **file handling**, and **basic image processing**. It serves as a simple and effective project for learning Python GUI development while creating a functional drawing application.

PROJECT OVERVIEW

The **Simple Drawing App** is a Python-based desktop application developed using the **Tkinter** library and **Pillow (PIL)**. It provides an interactive drawing canvas where users can create freehand sketches using the **Pencil**, erase unwanted parts with the **Eraser**, and draw **Lines, Rectangles, Circles, and Triangles** through a **Shapes dropdown menu**. The application allows users to select pen and canvas background colors, adjust the brush size, clear the canvas, and save drawings as **PNG** image files. It also includes a status bar that displays the selected tool, brush size, and pen color. This project demonstrates the practical use of **GUI programming**, **event handling**, **canvas graphics**, **file handling**, and **basic image processing**, making it an effective learning project for beginners in Python GUI development.

OBJECTIVES

- To develop a simple and user-friendly drawing application using **Python** and **Tkinter**.
- To understand the fundamentals of **GUI** and **event-driven programming**.
- To implement freehand drawing and basic shape drawing using mouse events.
- To provide drawing tools such as **Pencil, Eraser, Line, Rectangle, Circle,** and **Triangle**.
- To allow users to select **pen and canvas background colors**, adjust the brush size, clear the canvas, and save drawings as **PNG** image files.
- To enhance programming skills in **graphics programming**, **file handling**, and **Python GUI application development**.

S/W & H/W Requirements

Software Requirements:

- Operating System**:** Windows 10 / Windows 11
- Programming Language**:** Python
- Python Version**:** Python 3.x
- IDE**:** Visual Studio Code / PyCharm / IDLE
- GUI Library**:** Tkinter
- Additional Library**:** Pillow (PIL)

Hardware Requirements:

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Storage**:** 100 MB free disk space
- Display: 1366 × 768 resolution or higher
- Input Devices: Keyboard and Mouse/Touchpad

PROCESS

Step 1: Import the Required Libraries

Open **Visual Studio Code** and create a new Python file (for example, drawing_app.py). Import all the necessary libraries required for the application:

- **Tkinter** to create the graphical user interface (GUI).
- **ttk** from Tkinter to provide modern-looking widgets.
- **colorchooser** to allow users to select drawing colors.
- **filedialog** to let users choose the location for saving their drawings.
- **messagebox** to display success and error messages.
- **Pillow (PIL) – ImageGrab and Image** to capture and save the drawing canvas as a PNG image, with an alternative saving method using PostScript if required.
- **io** to handle the PostScript image data during the alternative save process.
- **ctypes** to enable DPI awareness, ensuring the application displays correctly on high-resolution screens.

Step 2: Configure DPI Awareness and Create the Main Window

Before creating the application window, enable **DPI awareness** using the ctypes library. This ensures that the application appears sharp and properly scaled on high-DPI displays.

Next, create the main Tkinter window by:

- Setting the window title to **"Simple Drawing App"**.
- Defining the window size as **1000 × 650 pixels**.

- Applying the **Clam theme** using ttk.Style() to give the interface a cleaner and more modern appearance.

Step 3: Initialize the Application Variables

Initialize all the variables required for drawing operations, including:

- The currently selected drawing tool (default: **Pencil**).
- The drawing color (default: **Black**).
- The brush size (default: **3 pixels**).
- The starting mouse coordinates for drawing shapes.
- The previous mouse coordinates for smooth freehand drawing.
- A preview object used to display temporary shapes while dragging the mouse.

These variables help manage drawing operations efficiently throughout the application.

Step 4: Design the Toolbar

Create a **vertical toolbar on the right side** of the application window containing buttons and controls for all drawing operations.

The toolbar includes:

- **Pencil** tool for freehand drawing.
- **Eraser** tool to erase drawings.

- **Shapes dropdown menu** containing Line, Rectangle, Circle, and Triangle tools.
- **Pen Color** button to choose the drawing color.
- **BG Color** button to change the canvas background color.
- **Vertical Brush Size** Slider to adjust the drawing thickness from 1 to 20 pixels.
- **Clear** button to remove all drawings from the canvas.
- **Save** button to save the artwork as a PNG image.

Step 5: Create the Drawing Canvas

Create a Canvas widget with a white background, crosshair cursor, and a subtle border, which serves as the main drawing area.

Step 6: Add a Status Bar

Create a status bar at the bottom of the application window using a StringVar.

The status bar continuously displays useful information such as:

- The currently selected drawing tool.
- The current brush size.
- The selected drawing color.

Whenever the user changes the tool, color, or brush size, the status bar is updated automatically.

Step 7: Bind Mouse Events

Connect mouse events to the canvas so users can interact with it naturally.

The following mouse events are used:

- **<Button-1>** records the starting coordinates when the left mouse button is pressed.
- **<B1-Motion>** continuously tracks mouse movement while the left button is held down, allowing drawing or live shape preview.
- **<ButtonRelease-1>** finalizes the selected shape when the mouse button is released.

These events provide a smooth and interactive drawing experience.

Step 8: Implement the Drawing Tools

Implement the functionality for each drawing tool.

- **Pencil:** Draws smooth freehand lines using the selected color and brush size.
- **Eraser:** Removes drawings by drawing with the **current canvas background color**, allowing it to work even after the background color is changed.
- **Line:** Draws a straight line between the starting and ending mouse positions.
- **Rectangle:** Draws a rectangular shape using the selected outline color and brush thickness.
- **Circle:** Draws an oval or circle between the starting and ending mouse positions.
- **Triangle:** Creates a triangle by calculating three points based on the mouse

position.

While drawing shapes, a temporary preview is displayed so users can see the shape before it is finalized.

Step 9: Implement Brush Size and Color Selection

Allow users to customize their drawings by providing:

- A **Pen Color Picker** to select the drawing color.
- A **Canvas Background Color Picker** to change the drawing area's background.
- A **Vertical Brush Size Slider** to adjust the thickness of the pencil, eraser, and shape outlines.

Whenever these settings change, the status bar is updated immediately.

Step 10: Implement the Clear Canvas Function

Create a Clear button that removes every object from the canvas using the delete("all") method and resets the canvas background color to white.

This feature allows users to quickly erase the entire drawing and start a new artwork without restarting the application.

Step 11: Implement the Save Drawing Function

Implement the save feature using PIL ImageGrab with a Canvas PostScript fallback.

The saving process includes:

- Opening a **Save File** dialog box for the user to choose the file name and save location.
- Updating the application window before capturing the drawing.
- Determining the canvas screen coordinates.
- Capturing the canvas using **ImageGrab** and saving it as a PNG image.
- If ImageGrab fails, using the Canvas PostScript method with Pillow as a fallback.
- Displaying a success or error message based on the result.

Step 12: Launch the Application

Finally, create an instance of the **DrawingApp** class and start the Tkinter event loop using root.mainloop().

Once the application starts, users can:

- Draw freehand sketches.
- Change the canvas background color.
- Select drawing shapes from the Shapes dropdown menu.
- Erase unwanted portions.
- Create straight lines, rectangles, circles, and triangles.

- Select custom colors.
- Adjust the brush size.
- View the selected tool, color, and brush size in the status bar.
- Clear the entire canvas.
- Save their artwork as a PNG image.

This completes the implementation of the **Simple Drawing Application**, providing an easy-to-use interface for creating, editing, and saving digital drawings.

SOURCE CODE

```
import tkinter as tk
from tkinter import ttk, colorchooser, filedialog, messagebox
from PIL import ImageGrab, Image
import ctypes
import io


# DPI Awareness (Windows)
try:
    ctypes.windll.shcore.SetProcessDpiAwareness(1)
except Exception:
    pass


class DrawingApp:
    def __init__(self, root):
        self.root = root
        root.title("Simple Drawing App")
        root.geometry("1000x650")
        root.configure(bg="#f0f0f0")


        self.tool = "Pencil"
        self.color = "#000000"
        self.size = 3
```

```
self.start_x = self.start_y = None
self.last_x = self.last_y = None
self.preview = None

style = ttk.Style()
try:
    style.theme_use("clam")
except Exception:
    pass

# ---------------- Main layout ----------------
# Added padding around the entire main frame
main = ttk.Frame(root, padding=10)
main.pack(fill="both", expand=True)

# Added a subtle gray border to the canvas and changed cursor
self.canvas = tk.Canvas(
    main, bg="white", cursor="crosshair",
    highlightthickness=2, highlightbackground="#cccccc"
)
self.canvas.pack(side="left", fill="both", expand=True, padx=(0, 10))

# Changed to LabelFrame to create a nice visual box around the tools
toolbar = ttk.LabelFrame(main, text=" Tools ", padding=10)
toolbar.pack(side="right", fill="y")
```

```
# Pencil
ttk.Button(toolbar, text="Pencil",
        command=lambda: self.set_tool("Pencil")).pack(fill="x", pady=3)

# Eraser
ttk.Button(toolbar, text="Eraser",
        command=lambda: self.set_tool("Eraser")).pack(fill="x", pady=3)

# ---------- Shapes Dropdown ----------
shape_var = tk.StringVar(value="Shapes")

shape_menu = ttk.OptionMenu(
    toolbar,
    shape_var,
    "Shapes",
    "Line",
    "Rectangle",
    "Circle",
    "Triangle",
    command=lambda value: self.set_tool(value)
)
shape_menu.pack(fill="x", pady=8)

# Foreground Color (Pen)
```

```
ttk.Button(toolbar, text="Pen Color",
        command=self.pick_color).pack(fill="x", pady=3)

# Background Color (Canvas) - NEW FEATURE ADDED HERE
ttk.Button(toolbar, text="BG Color",
        command=self.pick_bg_color).pack(fill="x", pady=3)

# Brush Size
ttk.Label(toolbar, text="Brush Size").pack(pady=(12, 2))

self.slider = ttk.Scale(
    toolbar,
    from_=1,
    to=20,
    orient="vertical",
    command=self.change_size,
    length=140
)
self.slider.set(3)
self.slider.pack(pady=5)

# Clear
ttk.Button(toolbar, text="Clear",
        command=self.clear).pack(fill="x", pady=3)
```

```
    # Save
    ttk.Button(toolbar, text="Save",
               command=self.save).pack(fill="x", pady=3)

    self.status = tk.StringVar()
    self.update_status()

    # Added slightly better styling to the status bar
    ttk.Label(root,
              textvariable=self.status,
              relief="sunken",
              padding=2,
              background="#e0e0e0",
              anchor="w").pack(fill="x", side="bottom")

    self.canvas.bind("<Button-1>", self.start)
    self.canvas.bind("<B1-Motion>", self.drag)
    self.canvas.bind("<ButtonRelease-1>", self.release)

def update_status(self):
    self.status.set(
        f" Tool: {self.tool}  |  Brush Size: {self.size}  |  Color: {self.color}"
    )

def set_tool(self, t):
```

```
    self.tool = t
    self.update_status()

def pick_color(self):
    c = colorchooser.askcolor(title="Choose Pen Color")[1]
    if c:
        self.color = c
        self.update_status()

# New method to handle Background Color selection
def pick_bg_color(self):
    c = colorchooser.askcolor(title="Choose Canvas Background Color")[1]
    if c:
        self.canvas.configure(bg=c)

def change_size(self, v):
    self.size = max(1, int(float(v)))
    self.update_status()

def start(self, e):
    self.start_x, self.start_y = e.x, e.y
    self.last_x, self.last_y = e.x, e.y

def drag(self, e):
    if self.tool in ("Pencil", "Eraser"):
```

```
    # If eraser, match the current canvas background color instead of hardcoded white
    col = self.canvas.cget("bg") if self.tool == "Eraser" else self.color
    self.canvas.create_line(
        self.last_x,
        self.last_y,
        e.x,
        e.y,
        fill=col,
        width=self.size,
        capstyle="round",
        smooth=True,
    )
    self.last_x, self.last_y = e.x, e.y
    return

if self.preview:
    self.canvas.delete(self.preview)

if self.tool == "Line":
    self.preview = self.canvas.create_line(
        self.start_x,
        self.start_y,
        e.x,
        e.y,
        fill=self.color,
```

```
            width=self.size,
        )

    elif self.tool == "Rectangle":
        self.preview = self.canvas.create_rectangle(
            self.start_x,
            self.start_y,
            e.x,
            e.y,
            outline=self.color,
            width=self.size,
        )

    elif self.tool == "Circle":
        self.preview = self.canvas.create_oval(
            self.start_x,
            self.start_y,
            e.x,
            e.y,
            outline=self.color,
            width=self.size,
        )

    elif self.tool == "Triangle":
        pts = [
```

```
            self.start_x,
            (self.start_y + e.y) / 2,
            (self.start_x + e.x) / 2,
            self.start_y,
            e.x,
            (self.start_y + e.y) / 2,
        ]
        self.preview = self.canvas.create_polygon(
            pts,
            outline=self.color,
            fill="",
            width=self.size,
        )

def release(self, e):
    if self.tool in ("Pencil", "Eraser"):
        return

    if self.preview:
        self.preview = None

def clear(self):
    self.canvas.delete("all")
    self.canvas.configure(bg="white") # This resets the background color to default!
```

```
def save(self):
    filepath = filedialog.asksaveasfilename(
        defaultextension=".png",
        filetypes=[("PNG", "*.png")]
    )

    if not filepath:
        return

    # Ensure the UI is up to date before capture
    self.root.update()

    x = self.canvas.winfo_rootx()
    y = self.canvas.winfo_rooty()
    x1 = x + self.canvas.winfo_width()
    y1 = y + self.canvas.winfo_height()

    # Try ImageGrab first (works on Windows and some platforms);
    # fall back to saving via canvas.postscript if needed.
    try:
        ImageGrab.grab((x, y, x1, y1)).save(filepath)
        messagebox.showinfo("Saved", "Drawing saved successfully!")
        return
    except Exception:
        pass
```

```
        try:
            # postscript approach (may require Ghostscript on some systems)
            ps = self.canvas.postscript(colormode="color")
            img = Image.open(io.BytesIO(ps.encode("utf-8")))
            img.save(filepath, "png")
            messagebox.showinfo("Saved", "Drawing saved successfully!")
        except Exception as ex:
            messagebox.showerror(
                "Error",
                  f"Could not save image: {ex}\n\nTry saving on Windows or install Ghostscript for postscript support."
            )


if __name__ == "__main__":
    root = tk.Tk()
    DrawingApp(root)
    root.mainloop()
```

OUTPUT

Figure 1: Initial Screen of the Simple Drawing Application

Description:

This figure shows the main interface of the **Simple Drawing Application** after it is launched. The application displays a blank white drawing canvas with a **vertical toolbar on the right side** containing the **Pencil**, **Eraser**, **Shapes** dropdown menu (Line, Rectangle, Circle, and Triangle), **Pen Color**, **BG Color**, **Brush Size** slider, **Clear**, and **Save** options. A status bar at the bottom displays the currently selected tool, brush size, and pen color.

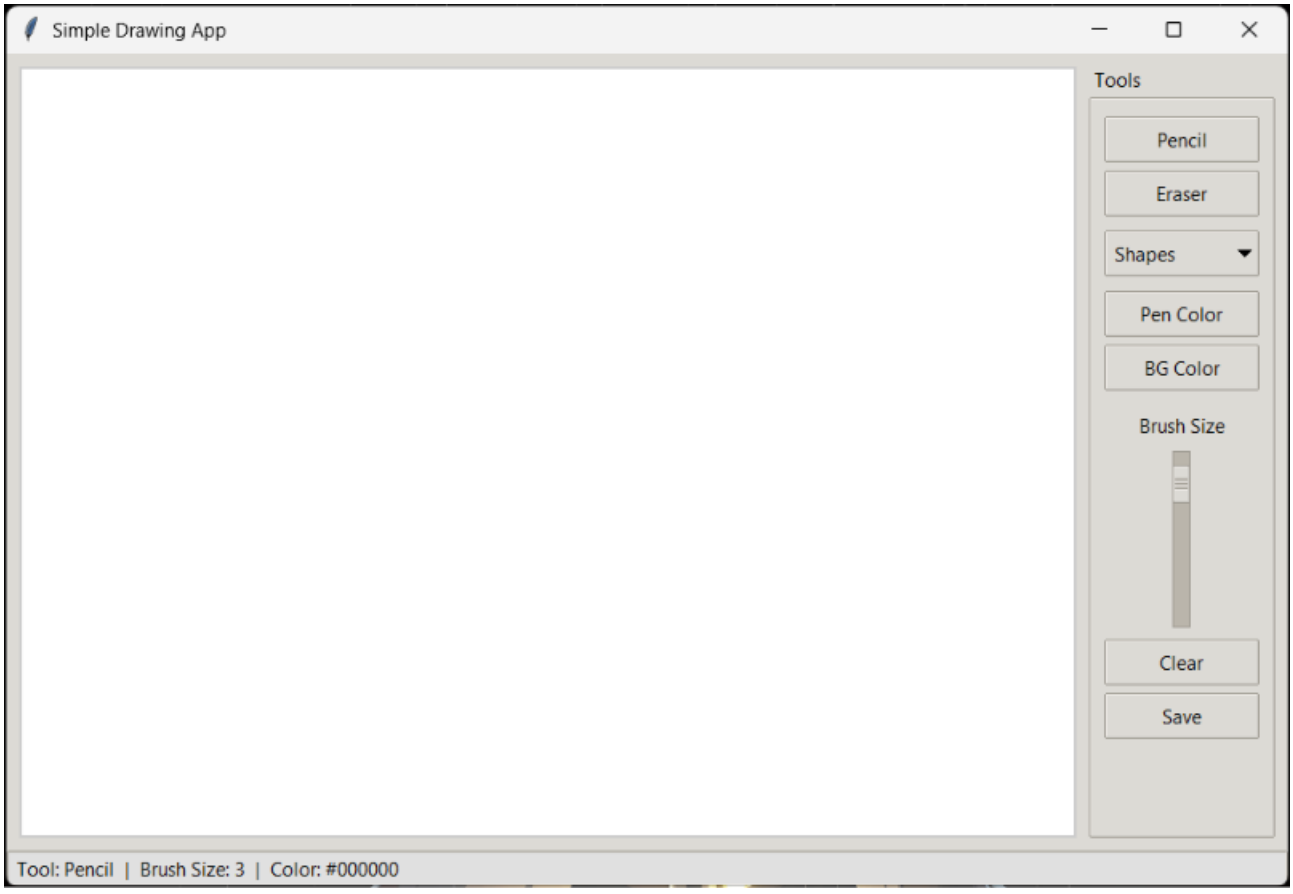


Figure 2: Freehand Drawing Using the Pencil Tool

Description:

This figure demonstrates drawing on the canvas using the **Pencil** tool. The user can freely create sketches and shapes by dragging the mouse over the canvas.

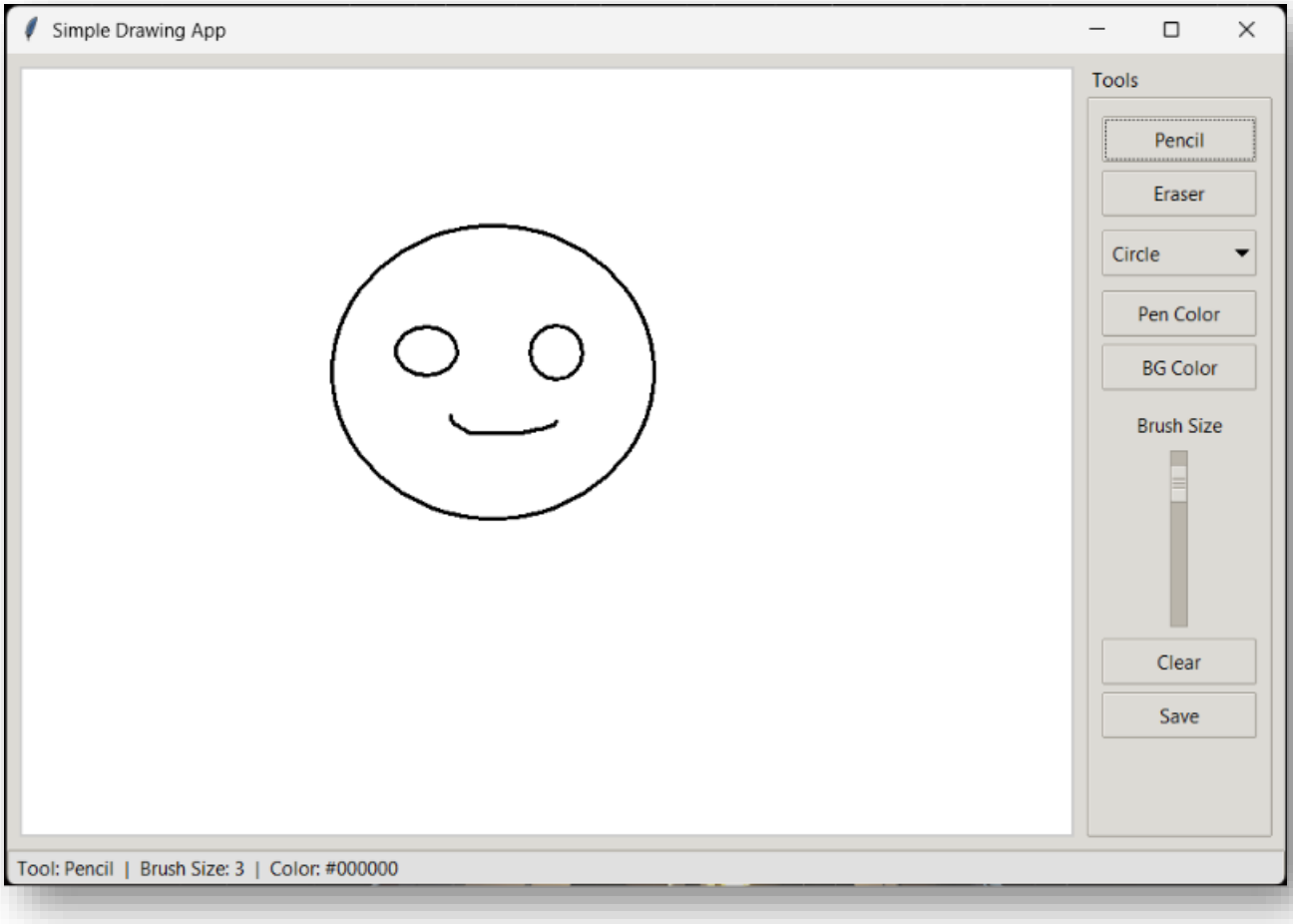


Figure 3: Drawing Geometric Shapes

Description:

This figure shows the creation of geometric shapes such as **Line, Rectangle, Circle,** and **Triangle**. Users can click and drag the mouse to draw the desired shape on the canvas.

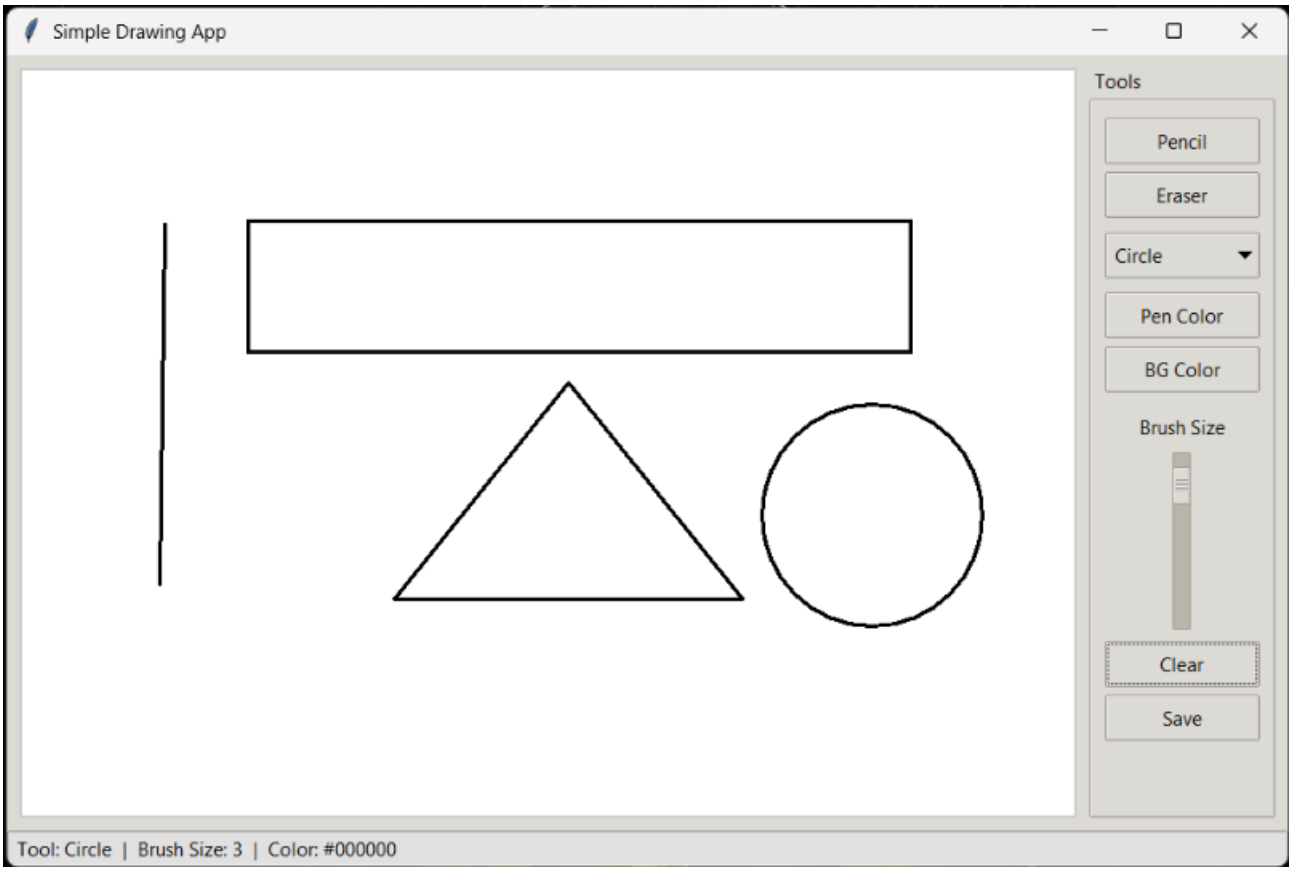


Figure 4: Using the Eraser Tool

Description:

This figure illustrates the **Eraser** tool removing selected portions of the drawing.

It helps users correct mistakes or modify their artwork easily.

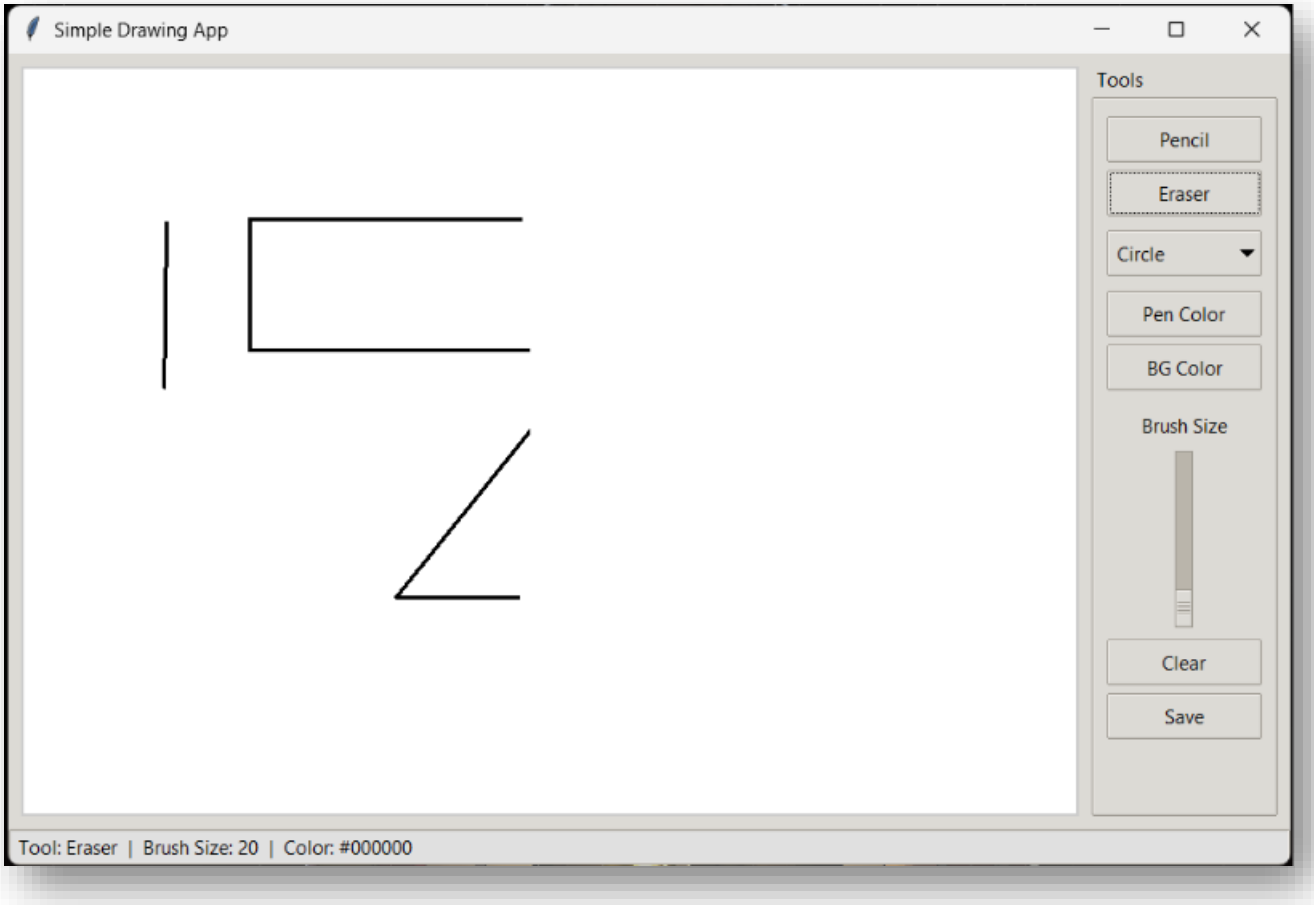


Figure 5: Changing the Pen Color and Background Color

Description:

This figure demonstrates the **Pen Color** and **BG Color** options available in the toolbar. Clicking either button opens the same **Color Picker** dialog. If the user selects **Pen Color**, the chosen color is applied to the drawing tools (such as the Pencil and Shapes). If the user selects **BG Color**, the selected color is applied to the canvas background. This allows users to customize both the drawing color and the background using the same color selection interface.

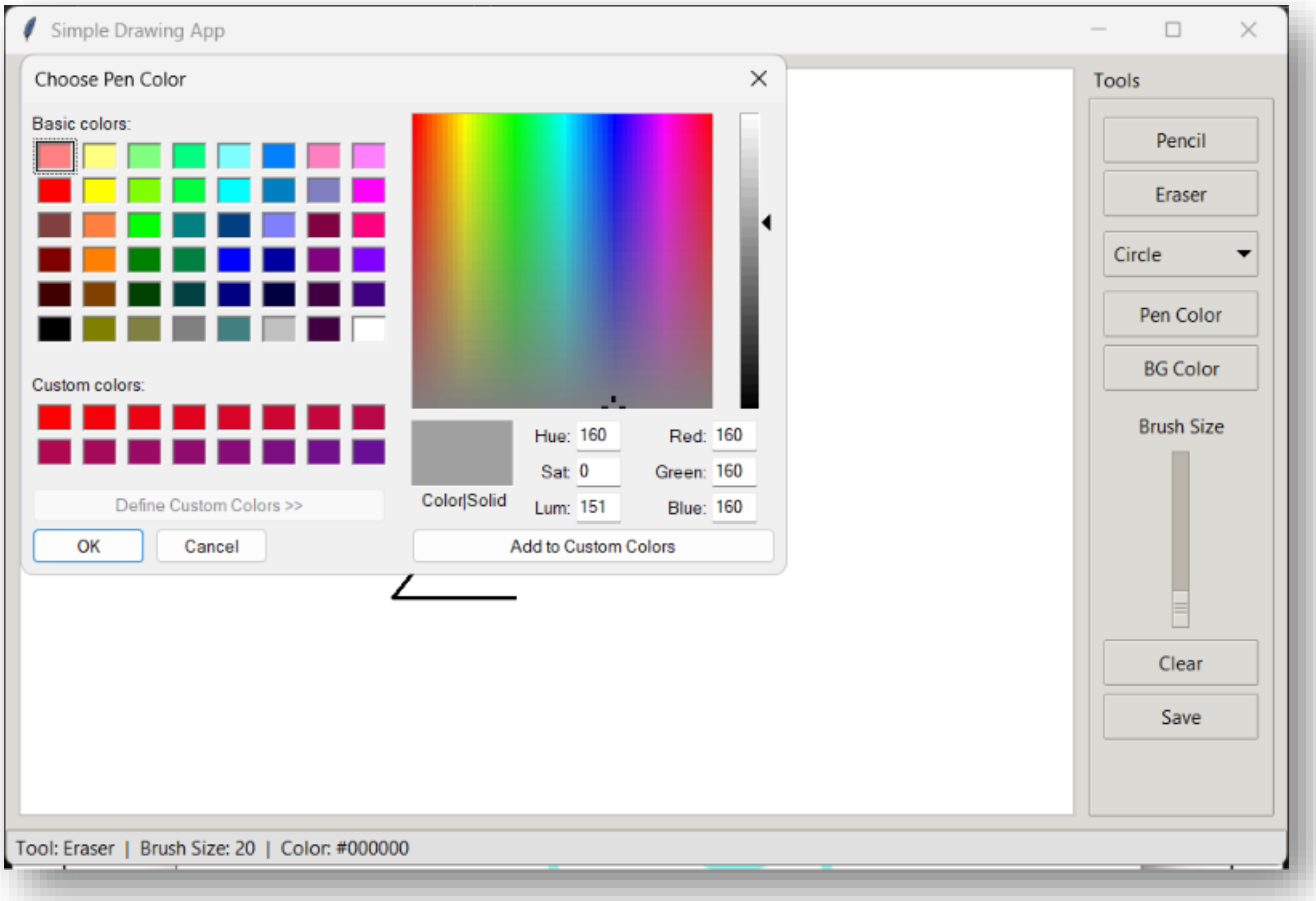


Figure 6: Adjusting the Brush Size and Clearing the Canvas

Description:

This figure demonstrates the **Brush Size** slider and the **Clear** option. Users can increase or decrease the thickness of the drawing strokes using the slider, while the **Clear** button removes all drawings from the canvas, providing a fresh workspace for creating a new drawing.

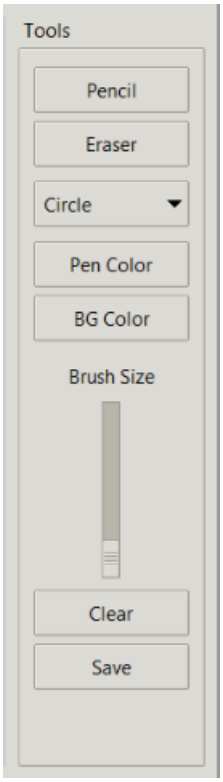


Figure 7: Saving the Drawing

Description:

This figure illustrates the **Save** feature. Users can save their completed drawing as a **PNG** image file by selecting the desired location on their computer.

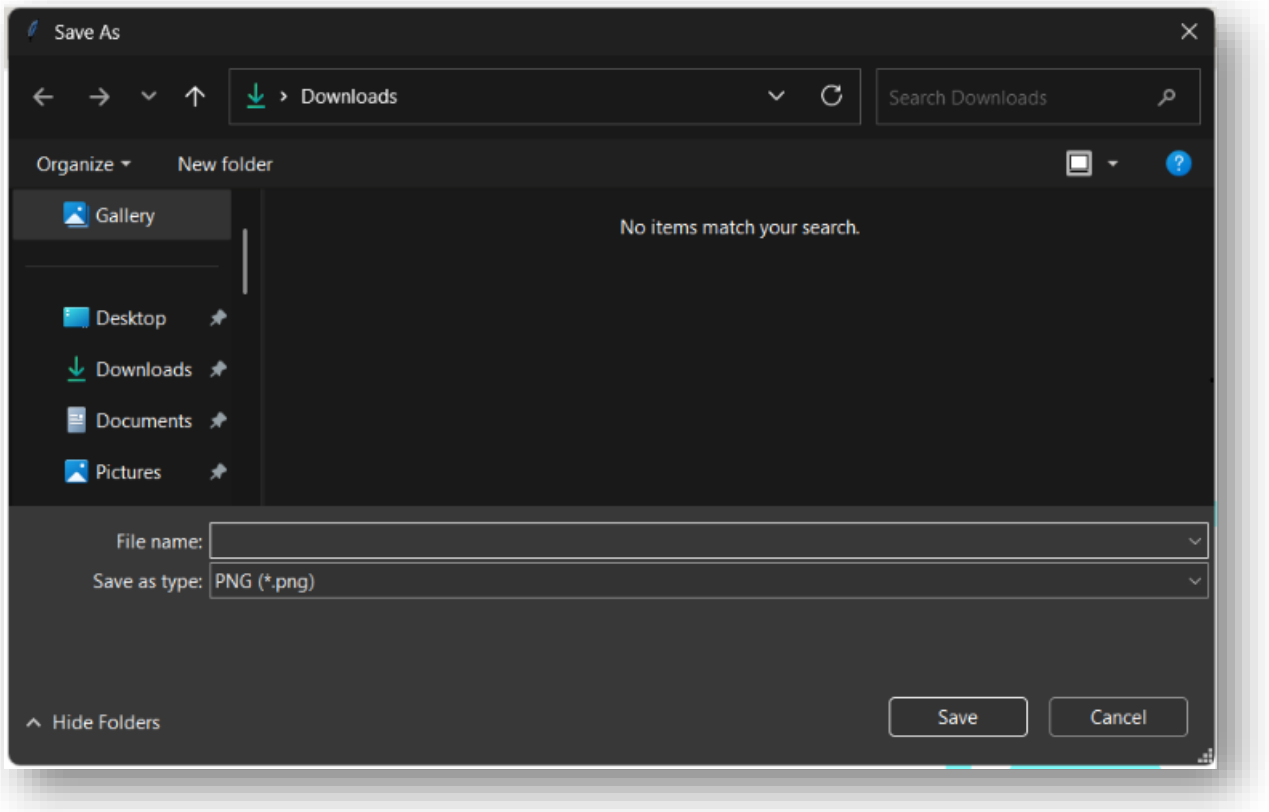
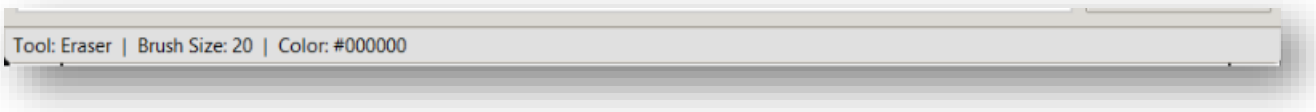


Figure 8: Status Bar Display

Description:

This figure shows the **Status Bar** at the bottom of the application. It displays the currently selected tool, brush size, and drawing color, providing real-time information to the user while drawing.



CONCLUSION

The **Simple Drawing App** was successfully developed using **Python**, **Tkinter**, and **Pillow (PIL)**. The application provides an easy-to-use interface where users can create freehand drawings, use the **Pencil** and **Eraser** tools, draw **Lines, Rectangles, Circles,** and **Triangles**, select pen and canvas background colors, adjust the brush size, clear the canvas, and save their artwork as **PNG** images.

This project demonstrates the practical implementation of **GUI programming**, **event-driven programming**, **canvas graphics**, **mouse event handling**, **file handling**, and **image processing** in Python. It serves as a good learning project for beginners and provides a strong foundation for developing more advanced desktop drawing applications. In the future, features such as **Undo/Redo**, **text insertion**, **fill options**, and **additional brush styles** can be added to further enhance the application's functionality.

FUTURE SCOPE

The **Simple Drawing App** can be enhanced by adding several advanced features to improve its functionality and user experience. Some possible future enhancements include:

- Implement **Undo** and **Redo** functionality to make editing drawings easier.
- Add a **Fill (Paint Bucket)** tool to fill enclosed areas with the selected color.
- Provide a **Text** tool for adding labels and notes to drawings.
- Introduce **multiple brush styles**, such as spray, calligraphy, and marker brushes.
- Add **Zoom In**, **Zoom Out**, and canvas navigation for detailed drawing.
- Support **opening and editing existing image files**.
- Allow saving drawings in additional formats such as **JPEG**, **BMP**, and **PDF**.
- Provide options to customize the **canvas size** and **background color**.
- Enhance the user interface by adding **icons**, **keyboard shortcuts**, and a modern theme.
- Improve the application with more advanced drawing and editing tools for a better user experience.

REFERENCES

- **Python Software Foundation.** ***Python 3 Documentation.***

  Available at: https://docs.python.org/3/

- **Python Software Foundation.** ***Tkinter.***

  Available at: https://docs.python.org/3/library/tkinter.html

- **Python Software Foundation.** ***ctypes Python.***

  Available at: https://docs.python.org/3/library/ctypes.html

- **Pillow Documentation.** ***Pillow (Python Imaging Library).***

  Available at: https://pillow.readthedocs.io/

- **GeeksforGeeks.** ***Python Tkinter Tutorial.***

  Available at: https://www.geeksforgeeks.org/python-tkinter-tutorial/

- **W3Schools.** ***Python Tkinter Tutorial.***

  Available at: https://www.w3schools.com/python/python_tkinter.asp

- **Youtube.** CodeWithHarry *Python Tkinter GUI Tutorial*

  Available at: https://youtu.be/-Q4lm8eYulw?si=P40LNPG0LOS4_pW1